Session 2 — The other path: from the neuron to ChatGPT

Artificial Intelligence · 4th year · UIR · Pr. Hakim Hafidi

Study sheet — exam material

1. The thread of this session

Session 1 followed people trying to *describe* intelligence precisely enough to program it, and watched that approach hit a wall: experts cannot write down what they know.

This session follows the other bet. Instead of describing intelligence, **copy the machine that produces it** — the brain — and let the rules grow from examples. The human stops writing rules and starts collecting data.

The path runs from a 1943 mathematical model of a neuron to ChatGPT in 2022. It is not a story of steady progress: it contains a twenty-year freeze caused by one counter-example, and a sudden take-off in 2012 that nobody in the field expected to be so abrupt. By the end you should be able to say what a neural network actually stores, why depth matters, and what three things had to arrive together before any of it worked.

2. From biological neuron to artificial neuron

The biological picture, simplified. A neuron receives electrical signals through its dendrites, sums them in the cell body, and fires along its axon if the total crosses a threshold. Connections between neurons — synapses — vary in strength, and that strength changes with use. The human brain holds on the order of 86 billion neurons and perhaps 100 trillion synapses.

McCulloch and Pitts (1943). Warren McCulloch, a neurophysiologist, and Walter Pitts, a logician, published *A Logical Calculus of the Ideas Immanent in Nervous Activity*. They stripped the neuron down to arithmetic: inputs, each multiplied by a weight; a sum; a threshold; an output of 0 or 1. They then proved that networks of such units can compute any logical function. This is the founding result of the connectionist path — and notice that it arrives thirteen years before the word "artificial intelligence" exists.

What the artificial neuron does, precisely. Take the inputs x_1, x_2, … x_n. Multiply each by its weight w_1, w_2, … w_n. Add them up, add a bias term, and pass the result through a function that decides the output. That is the entire computation. Everything else in deep learning is this operation repeated, arranged and scaled.

The weights are the knowledge. This is the sentence to carry out of the session. A trained network does not store examples, rules or facts in any form a human can read. It stores numbers on connections. Learning means adjusting those numbers. When we say GPT-3 has 175 billion parameters, we are saying it has 175 billion such numbers, and that all of what it "knows" is encoded in them.

Hebb's rule (1949). Donald Hebb proposed a mechanism for how biological learning might work: "neurons that fire together, wire together". Connections used simultaneously are strengthened. This is the first concrete proposal for *how* weights could change from experience rather than being set by a designer.

3. The perceptron and the twenty-year freeze

Rosenblatt's perceptron (1958). Frank Rosenblatt, at Cornell, built a machine that learned. The Mark I Perceptron was a physical device with photocells for input and motorised potentiometers for weights, designed to recognise simple shapes. Its learning rule is almost trivially simple: show it an example; if the output is wrong, nudge the weights in the direction that would have made it right; repeat.

What made this historic is that **nobody wrote the recognition rule**. The machine derived it from examples. The *New York Times* reported in 1958 that the Navy expected "the embryo of an electronic computer that will be able to walk, talk, see, write, reproduce itself and be conscious of its existence". The gap between that sentence and the machine's actual ability is the pattern Session 1 called over-promising.

The XOR problem (1969). In *Perceptrons*, Marvin Minsky and Seymour Papert analysed precisely what a single-layer perceptron can and cannot learn. Their central counter-example is the exclusive-or function: output 1 when exactly one of two inputs is 1.

The reason it fails is geometric and worth understanding, because it recurs throughout the course. A single neuron computes a weighted sum and compares it to a threshold; geometrically, that draws **one straight line** through the input space and answers on one side of it. XOR places its two positive cases diagonally opposite each other, and no single straight line separates them from the two negative cases. The limit is not about computing power or training time: it is about the shape of what one neuron can express.

The freeze. The result was known, but the book's influence was institutional. Funding for neural network research largely stopped; the symbolic school had the field for the next fifteen years. This matters as a lesson in how fields move: a correct technical result about *one-layer* networks was widely read as a verdict on the whole approach.

The fix that was already known. Stacking layers solves XOR — a hidden layer can build intermediate features that make the problem separable. The obstacle was that nobody had a practical method to train the weights of a layer that is not directly connected to the output. How do you assign blame to a neuron in the middle?

4. Backpropagation and what "deep" means

Backpropagation (1986). Rumelhart, Hinton and Williams popularised an answer: propagate the error *backwards* through the network, so that each weight receives its share of responsibility for the mistake. Each weight is then adjusted a little in the direction that reduces the error. Repeat over millions of examples and the whole network — every layer — learns together.

The principle is the one Session 3 formalises as gradient descent: the error is a landscape, backpropagation computes the slope under your feet for every parameter at once, and training is a long walk downhill.

Hidden layers and what they build. A layer between input and output is called hidden because nothing outside the network sees it. Its job is to construct intermediate representations. In a network trained on photographs, the first layers respond to edges and colour gradients; middle layers to corners, textures and simple shapes; deeper layers to object parts — an eye, a wheel, a letter. Nobody designed this hierarchy. It emerges because it reduces the error.

Representation learning: the real break with symbolic AI. In classical machine learning, a human expert designs the *features*: for face recognition, someone decides to measure the distance between the eyes, the width of the nose, and so on. The algorithm then weighs those features. In deep learning, the network discovers its own features from raw pixels.

This shifts the expert's job from *describe the world* to *collect and label the data*. It is the direct answer to the knowledge-acquisition bottleneck of Session 1 — and it explains why data became the strategic asset it is today.

	Symbolic AI	Classical ML	Deep learning
Who writes the rules	Human	Algorithm, from data	Algorithm, from data
Who designs the features	Human	Human	The network itself
What happens when the world changes	Rewrite rules	Retrain on new data	Retrain on new data
Explainability	High — rules are readable	Moderate	Low
Data needed	None	Moderate	Very large

5. Why 2012 and not 1986

Backpropagation worked in 1986. Deep learning took off in 2012. Twenty-six years passed, and understanding why is the most useful piece of history in this session: **three ingredients had to arrive together**, and none of them is an idea.

Data. Supervised learning needs labelled examples in quantity. Fei-Fei Li's **ImageNet**, released in 2009, provided roughly 14 million hand-labelled images across 20,000 categories — an effort of a scale nobody had attempted. The web supplied text at a comparable scale.

Compute. Graphics processors, built to render video games, turn out to perform exactly the operation neural networks need: the same arithmetic applied to very many numbers in parallel. A GPU could train a network tens of times faster than a CPU, turning a month of computation into a day.

Algorithms. Incremental but decisive improvements — better activation functions, better weight initialisation, regularisation methods such as dropout — made deep networks trainable in practice rather than just in principle.

AlexNet, 2012. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton entered the ImageNet competition with a deep convolutional network trained on two consumer GPUs. It cut the top-5 error rate from about 26% to about 16% — a margin so far beyond the usual yearly improvement that the result was initially met with scepticism. Within two years, essentially every entrant used deep learning. This is the moment the current era begins.

The scale since. Training compute for frontier models has grown by a factor of roughly 200 million between AlexNet and today. **Scaling laws**, described systematically by Kaplan and colleagues in 2020, showed that performance improves smoothly and predictably as model size, data and compute grow — which is why laboratories kept building bigger models rather than waiting for better ideas.

6. Three architectures and the one that won

Convolutional neural networks (CNNs). Designed for images. Instead of connecting every pixel to every neuron, a small filter slides across the image looking for a local pattern — the same filter everywhere, because an edge is an edge wherever it appears. Yann LeCun's **LeNet** (1998) read handwritten digits on bank cheques; AlexNet is a deeper descendant.

Recurrent neural networks (RNNs). Designed for sequences. The network reads one element at a time and carries forward a summary of what it has seen. This works, but each step overwrites part of the memory, so information from far back fades — and because reading is sequential, training cannot be parallelised across the sequence. Both limitations are fatal at scale.

The Transformer (2017). The paper *Attention Is All You Need* (Vaswani et al., Google) replaced sequential reading with **attention**: every element looks at every other element simultaneously and weighs how relevant each is. Two consequences. Distance stops mattering — a word can attend directly to a word a thousand positions earlier. And the whole sequence is processed in parallel, which is what makes training on GPUs at scale possible.

Every large language model in use today is a Transformer. Session 4 takes it apart.

7. From the Transformer to ChatGPT

GPT-3 (2020). 175 billion parameters, trained on hundreds of billions of tokens of text, with a single objective: predict the next token. It could write essays, translate, and produce working code — none of which it was explicitly trained to do. The capability came from scale plus one objective.

The gap GPT-3 left. It completed text; it did not answer questions. Asked "explain photosynthesis", it might continue with more exam questions, because that is what such text looks like in its training data. Turning a completer into an assistant required an extra training stage, which is Session 5.

ChatGPT (30 November 2022). A fine-tuned model plus a chat interface plus training on human feedback. It reached 100 million users in about two months — the fastest consumer software adoption recorded at the time. Nothing fundamental changed in the architecture; what changed was the interface and the alignment training.

Generative adversarial networks (2014). Worth knowing as the other branch of generative AI. Ian Goodfellow's GANs pit two networks against each other — a generator producing fakes and a discriminator trying to detect them — each improving the other. They produced the first convincing synthetic faces.

AlphaGo (2016). DeepMind's system beat Lee Sedol at Go, a game with more legal positions than atoms in the universe and no usable evaluation function. It combined deep networks with search, and learned partly by playing against itself. It is the clearest demonstration that the symbolic tradition (search) and the connectionist tradition (learned evaluation) are complementary rather than opposed.

Recognition (2024). Geoffrey Hinton and John Hopfield received the Nobel Prize in Physics for foundational work on neural networks — an acknowledgement, decades later, of research that was unfundable in the 1970s.

8. Hype cycles: how to read a claim

AI has moved in waves since 1956, and the shape repeats: a genuine breakthrough, then over-extrapolation from it, then money, then contact with reality, then collapse, then the field renames itself and the next wave begins.

The useful habit is not cynicism but specificity. When you read a claim, ask: what exactly was measured, on what data, compared with what baseline, and does the result hold outside the conditions of the demonstration? Those four questions are the subject of Lab 4, and they are what separates an engineer from an enthusiast.

9. Key terms

Term	Meaning
Artificial neuron	Weighted inputs → sum → threshold → output. The weights are the knowledge.
McCulloch–Pitts model	The 1943 mathematical neuron; networks of them can compute any logical function.
Hebb's rule	"Neurons that fire together, wire together" (1949): connections used together are strengthened.
Perceptron	Rosenblatt's 1958 learning neuron, whose weights adjust from its own mistakes.
XOR problem	A single-layer perceptron draws one straight line and cannot express "one or the other but not both" (Minsky & Papert, 1969).
Hidden layer	A layer between input and output; needed for XOR; where intermediate features are built.
Backpropagation	Propagating the output error backwards so every weight in every layer receives its share of the blame (1986).
Deep learning	Neural networks with many layers that learn their own features.
Representation learning	Features discovered from data rather than designed by an expert.
CNN	Convolutional network: small filters slide across an image looking for local patterns (LeNet 1998, AlexNet 2012).
RNN	Recurrent network: reads a sequence one step at a time, carrying a memory forward. Forgets long contexts; trains sequentially.
Transformer	The 2017 architecture based on attention; processes a whole sequence in parallel. Behind every modern LLM.
Attention	A mechanism letting each element weigh all others when computing its representation, regardless of distance.
ImageNet	~14 million labelled images (2009); the benchmark on which AlexNet cut the error from 26% to 16%.
Scaling laws	Performance improves predictably with model size, data and compute (Kaplan et al., 2020).
GAN	Generator and discriminator trained against each other (Goodfellow, 2014).
Hype cycle	Breakthrough → over-extrapolation → funding → reality → collapse → rebrand → next wave.

10. Dates and numbers to know

1943 McCulloch & Pitts · 1949 Hebb's rule · 1958 perceptron · **1969** ***Perceptrons, the XOR limit · 1986 backpropagation popularised · 1998 LeNet · 2009 ImageNet (~14 million images) · 2012 AlexNet: top-5 error 26% → 16% · 2014 GANs · 2016 AlphaGo beats Lee Sedol · 2017 Transformer, Attention Is All You Need*** · 2020 GPT-3 (175 billion parameters) · 2020 scaling laws · **30 November 2022 ChatGPT, 100 million users in two months** · 2024 Nobel Prize in Physics to Hinton and Hopfield · brain: ~86 billion neurons, ~100 trillion synapses · training compute growth since AlexNet: ~200 million-fold.

11. Common misconceptions

"Neural networks work like the brain." They were inspired by a 1943 simplification of a neuron. A biological neuron is vastly more complex, learning in the brain does not use backpropagation, and the brain runs on about 20 watts. The analogy is historical, not mechanical.

"Deep learning was invented in 2012." The algorithms date from the 1980s. What arrived in 2012 was data and compute. This is why "we need a new idea" is often the wrong diagnosis.

"More layers always means better." Depth helps only with enough data and the right training techniques. Lab 3 and Lab 4 show a large network memorising a small dataset instead of learning it.

"The network understands what it sees." It computes a function that maps inputs to outputs with low error on data resembling its training set. Lab 4 is built entirely on the gap between that and understanding.

12. Pause & recall — answers

I. (a) Learning consists in strengthening the connections used together — Hebb's rule. (b) Weighted inputs, a sum, and a threshold. (c) Weights that adjust themselves from mistakes: the machine learns the rule rather than being given it.

II. (a) Points that no single straight line can separate; it exposed the limit of one-layer networks and froze funding for the approach. (b) It propagates the output error backwards so that every weight in every layer can be adjusted. (c) Computers were too slow and no large labelled

dataset existed.

III. (a) In the weights — the numbers on the connections. (b) Many hidden layers, each building more abstract features from the one below. (c) Designed features are written by an expert; learned features are discovered by the network from raw data.

IV. (a) Data (ImageNet), compute (GPUs) and algorithms (backpropagation at scale, then the Transformer). (b) AlexNet, a deep CNN trained on GPUs, cut the top-5 error from 26% to 16%, and within two years the whole field had switched. (c) Every element attends to every other element at once instead of being read in order, so distance stops mattering and training runs in parallel.

13. Sample exam questions

1. Compare the symbolic and connectionist approaches on three points: who writes the knowledge, what happens when the world changes, and explainability.
2. Explain what a hidden layer is and why the XOR problem requires one. A diagram is welcome.
3. List the three ingredients of the 2012 take-off, give one example of each, and explain why backpropagation alone was not enough in 1986.
4. What does "representation learning" mean, and in what sense does it answer the knowledge-acquisition bottleneck of Session 1?
5. Why did the Transformer replace recurrent networks? Give two distinct reasons.